Teoria Współbieżności

Kajetan Frątczak 09.10.2025 r.

Laboratorium 01

Wyścig wątków

1. Treść zadań

Zadanie 1

1.1 Napisać program (szkielet), który uruchamia 2 wątki, z których jeden zwiększa wartość zmiennej całkowitej o 1, drugi wątek zmniejsza wartość o 1. Zakładając że na początku wartość zmiennej Counter była 0, chcielibyśmy wiedzieć jaka będzie wartość tej zmiennej po wykonaniu 100000 operacji zwiększania i zmniejszania przez obydwa wątki.

1.2 Na podstawie 100 wykonań programu z p.1.1, stworzyć histogram końcowych wartości zmiennej Counter.

1.3 Spróbować wprowadzić mechanizm do programu z p.1, który zagwarantowałby przewidywalną końcową wartość zmiennej Counter. Nie używać żadnych systemowych mechanizmów (np. synchronized), tylko swój autorski.

Zadanie 2

W systemie działa N wątków, które dzielą obiekt licznika (początkowy stan licznika = 0).

Każdy wątek wykonuje w pętli 5 razy inkrementację licznika. Zakładamy, że inkrementacja składa się z sekwencji trzech instrukcji: read, inc, write (odczyt z pamięci, zwiększenie o 1, zapis do pamięci). Wątki nie są synchronizowane.

2.1 Jaka jest teoretycznie najmniejsza wartość licznika po zakończeniu działania wszystkich wątków i jaka kolejność instrukcji (przeplot) do niej prowadzi?

2.2 Spróbować znaleźć dowód, że będzie to zawsze najmniejsza wartość.

2. Argumentacja rozwiązania

Zadanie 1.1

W tym zadaniu uruchomiłem dwa wątki, które współdzielą obiekt Counter. Jeden wątek wykonuje 100 000 inkrementacji, drugi 100 000 dekrementacji. Z powodu braku synchronizacji wynik końcowy nie jest deterministycznie równy 0 – zależy od tego, jak system przeplata instrukcje.

Zadanie 1.2

Program uruchamiam 100 razy, aby zobaczyć jak zmienia się wartość licznika. Wyniki powinny być równe 0, ale występują odchylenia zarówno dodatnie, jak i ujemne.

Zadanie 1.3

Aby zapewnić deterministyczny wynik (zawsze 0), należałoby wprowadzić własny mechanizm synchronizacji. Zdecydowałem się wprowadzić mechanizm synchronizacji korzystając z algorytmu Petersona. Jest to klasyczny algorytm do osiągnięcia wzajemnego wykluczenia dla dwóch procesów (w tym przypadku wątków).. Algorytm wykorzystuje dwie zmienne: tablicę flag oraz zmienną turn, które pozwalają wątkom na koordynację dostępu do sekcji krytycznej.

Zadanie 2.1

Mamy N wątków wykonujących po 5 inkrementacji. Każda inkrementacja to sekwencja: read (odczyt wartości), inc (zwiększenie o 1), write (zapis wartości). Z początku myślałem, że najmniejsza wartość licznika wynosi 5, jednak po rozpisaniu odpowiedniego przeplotu byłem w stanie uzyskać stan licznika równy 2.. Aby osiągnąć tę wartość, musi dojść do sytuacji, w której wątki sczytają wartość sprzed inkrementacji (będzie to coś w stylu nadpisywania wartości).

Zadanie 2.2

Chcę udowodnić, że wartość 2 jest zawsze najmniejszą możliwą wartością końcową licznika.

3. Rozwiązanie

```java
class Counter {
   private int _val;
   public Counter(int n) {
       _val = n;
   }
   public void inc() {
       _val++;
   }
   public void dec() {
       _val--;
   }
   public int value() {
       return _val;
   }
}

// Watek, ktory inkrementuje licznik 100.000 razy
class IThread extends Thread {
   private Counter _counter;
   public IThread(Counter counter) {
       _counter = counter;
   }
   @Override
   public void run() {
       for (int i = 0; i < 100000; i++) {
           _counter.inc();
       }
   }
}

// Watek, ktory dekrementuje licznik 100.000 razy
class DThread extends Thread {
   private Counter _counter;
   public DThread(Counter counter) {
       _counter = counter;
   }
   @Override
   public void run() {
       for (int i = 0; i < 100000; i++) {
           _counter.dec();
       }
   }
}

public class Race {
   public static void main(String[] args) {
       for (int i = 0; i < 100; i++) {
           Counter cnt = new Counter(0);

           Thread t1 = new IThread(cnt);
           Thread t2 = new DThread(cnt);

           t1.start();
           t2.start();

           try{
               t1.join();
               t2.join();
           } catch (InterruptedException e) {
               e.printStackTrace();
           }

           System.out.println("stan=" + cnt.value());
       }
   }
}
```

4. Wykresy, tabele, wyniki

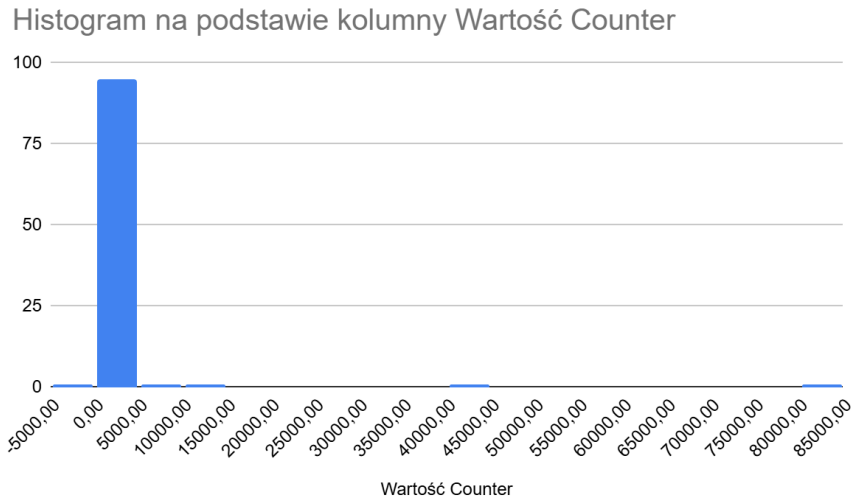
Na poniższym screenie można zobaczyć fragment wyników, jakie otrzymałem.

```
stan=11190
stan=-4672
stan=40914
stan=4886
stan=7736
stan=84756
stan=0
stan=0
stan=0
stan=0
stan=0
stan=0
```

Rysunek 1: Wynik po uruchomieniu programu

Co ciekawe, po pewnym czasie wartość licznika jest już ciągle równa 0. Jest to spowodowane optymalizacją JIT (Just-In-Time) kompilatora w JVM. Z początku kod działa w trybie interpretowanym, jednak po pewnym czasie (po kilku wywołaniach) JVM wykrywa, że metody dec() oraz inc() są często wywoływane i JIT kompilator optymalizuje kod.

Histogram wyników wklejam poniżej.



Rysunek 2: Histogram wyników

Jako mechanizm synchronizacji wątków dodałem algorytm Petersona, który zapewnia wzajemne wykluczenie (mutual exclusion), umożliwiające dwóm procesom lub wątkom (jak w naszym przypadku) bezkonfliktowy dostęp do współdzielonego zasobu (sekcji krytycznej).

```java
class Counter {
   private int _val;
   private PetersonLock _lock;

   public Counter(int n) {
       _val = n;
       _lock = new PetersonLock();
   }
   public void inc(int threadId) {
       _lock.lock(threadId);
       _val++;
       _lock.unlock(threadId);
   }
   public void dec(int threadId) {
       _lock.lock(threadId);
       _val--;
       _lock.unlock(threadId);
   }
   public int value() {
       return _val;
   }
}

class PetersonLock {
   private volatile boolean[] flag = new boolean[2];
   private volatile int turn;
   public PetersonLock() {
       flag[0] = false;
       flag[1] = false;
       turn = 0;
   }

   public void lock(int threadId) {
       int other = 1 - threadId;
       flag[threadId] = true;
       turn = other;
       while (flag[other] && turn == other) {
           // busy waiting
       }
   }

   public void unlock(int threadId) {
       flag[threadId] = false;
   }
}

// Watek, ktory inkrementuje licznik 100.000 razy
class IThread extends Thread {
   private Counter _counter;
   private static final int THREAD_ID = 0;
   public IThread(Counter counter) {
       _counter = counter;
   }
   @Override
   public void run() {
       for (int i = 0; i < 100000; i++) {
           _counter.inc(THREAD_ID);
       }
   }
}
```

```java
// Watek, ktory dekrementuje licznik 100.000 razy
class DThread extends Thread {
   private Counter _counter;
   private static final int THREAD_ID = 1;
   public DThread(Counter counter) {
       _counter = counter;
   }
   @Override
   public void run() {
       for (int i = 0; i < 100000; i++) {
           _counter.dec(THREAD_ID);
       }
   }
}

public class Race {
   public static void main(String[] args) {
       for (int i = 0; i < 100; i++) {
           Counter cnt = new Counter(0);

           Thread t1 = new IThread(cnt);
           Thread t2 = new DThread(cnt);

           t1.start();
           t2.start();

           try{
               t1.join();
               t2.join();
           } catch (InterruptedException e) {
               e.printStackTrace();
           }

           System.out.println("stan=" + cnt.value());
       }
   }
}
```

```
stan=0
stan=0
stan=0
stan=0
stan=0
stan=0
stan=0
stan=0
stan=0
stan=0
```

Rysunek 3: Wyniki po wprowadzeniu synchronizacji

Zadanie 2 - rozwiązanie (niepoprawne - 1 próba)

Zad. 2

N wątków dzieli dostęp licznika (początkowy stan licznika = 0)

Każdy wątek wykonuje w pętli 5 razy inkrementację licznika

inkrementacja składa się z trzech instrukcji: read, inc, write

2.1.

Jaka jest teoretycznie najmniejsza wartość licznika po zakończeniu działania wszystkich wątków i jaka kolejność instrukcji (przeplot) do niej prowadzi?

Pomimo wykonać się 5·N inkrementacji, jednak przez wyścig wątków spowodowany brakiem synchronizacji, wynik końcowy może być mniejszy niż 5N (np. dwa wątki odczytają tę samą wartość licznika i oba zapiszą tę samą wartość powiększoną o 1)

Co się może stać?

① Wszystkie wątki mogą zadziałać jednocześnie i wtedy stan licznika wyniesie 5.

Przykład (scenariusz minimalny) -> wątki działają „po kolei" (wątek nie robi read, gdy inny nie zdążył wykonać write)

N = 3

wątki: T1, T2, T3

stan licznika: 0

1) wątek T1 wykonuje operację read (licznik_T1 = 0)
   wątek T2 wykonuje operację read (licznik_T2 = 0)
   wątek T3 wykonuje operację read (licznik_T3 = 0)
   } wszystkie wątki czytają 0

następnie T1 wykonuje inc oraz write (licznik_T1 = 1) -> stan licznika = 1

następnie T2 wykonuje inc oraz write (licznik_T2 = 1) -> stan licznika = 1

następnie T3 wykonuje inc oraz write (licznik_T3 = 1) -> stan licznika = 1

zatem stan licznika po 1 wykonaniu w pętli = 1

2), 3), 4), 5) -> analogicznie zatem ostatecznie stan licznika = 5

Rysunek 4: Rozwiązanie zadania 2

Przykład (scenariusz maksymalny) -> wątki działają „na raz" (wszystkie robią read na tej samej wartości zanim którykolwiek wykona write)

$N = 3$

wątki: T1, T2, T3

stan licznika: 0

Wątek T1

1) wątek T1 wykonuje read, inc, write
2) wątek T1 wykonuje read, inc, write
3), 4), 5) analogicznie -> stan licznika = 5

Wątek T2

1) wątek T2 wykonuje read, inc, write
2) wątek T2 wykonuje read, inc, write
3), 4), 5) analogicznie -> stan licznika = 10

Wątek T3

1) wątek T3 wykonuje read, inc, write
2) wątek T3 wykonuje read, inc, write
3), 4), 5) analogicznie -> stan licznika = 15

Przeplot minimalny:

$(r_1, r_2, r_3, \dots, r_N, i_1, w_1, i_2, w_2, \dots, i_N, w_N)^5$

Przeplot maksymalny:

$(r_1, i_1, w_1)^5, (r_2, i_2, w_2)^5, \dots, (r_N, i_N, w_N)^5$

r, i, w – read, inc, write

2.2

Dowód

N wątków wykonuje w pętli 5 razy inkrementację licznika składającą się z operacji read, inc, write

zatem mamy do wykorzystania operacje: $(r_1, \dots r_N, i_1, \dots, i_N, w_1, \dots, w_N)^5$

nawet gdy operacje $i_1, \dots, i_N$ będą wykonywane na tym samym stanie licznika, to mamy ich 5 razy tyle, zatem musimy zwiększyć stan licznika o co najmniej 5 CKD

Rysunek 5: Rozwiązanie zadania 2 cd.

Zadanie 2 - rozwiązanie (poprawne, bez formalnego dowodu)

- Powyższe podejście nie jest jednak minimalne -> stan licznika możemy zminimalizować aż do 2 przy odpowiednim przeplocie instrukcji.

<u>Przykład</u>

$N = 2$

wątki : T1, T2

stan licznika : $S = 0$

<u>Przeplot:</u>

$$\underbrace{r_1, i_1}_{(1)}, \underbrace{r_2, i_2}_{(1)}, \underset{\downarrow \atop 1}{w_1}, \underbrace{(r_1, i_1, w_1)}_{(4)}{}^{3}, \underset{\downarrow \atop 1}{w_2}, \underbrace{r_1, i_1}_{(2)}, \underbrace{(r_2, i_2, w_2)}_{(5)}{}^{4}, \underbrace{w_1}_{2}$$

końcowy stan licznika = 2

Rysunek 6: Rozwiązanie zadania 2 cd.

5. Wnioski

- Kod nieposiadający mechanizmu synchronizacji zachowuje się w sposób niedeterministyczny.
- Algorytm Petersona jest skuteczny, ale ma ograniczenia. Implementacja algorytmu Petersona zagwarantowała deterministyczny wynik (zawsze 0) we wszystkich 100 próbach. Jednakże algorytm ten działa tylko dla dwóch wątków i wykorzystuje aktywne oczekiwanie (busy waiting), co nie jest efektywne w kontekście zużycia procesora - program działa dłużej.
- Mechanizmy synchronizacji oferowane przez Javę są lepsze niż autorskie rozwiązania. Choć implementacja własnego mechanizmu jest pouczająca, w praktyce korzystanie z gotowych, przetestowanych rozwiązań oferowanych przez Javę, np. *synchronized* jest bardziej wydajne.

6. Bibliografia

https://pl.wikipedia.org/wiki/Algorytm_Petersona
https://www.geeksforgeeks.org/dsa/petersons-algorithm-in-process-synchronization/